Injection flaws

Injection flaws let attackers relay malicious code through a Web application to another system. These attacks include calls to the OS via system calls, the use of external programs via shell commands, and calls to back-end databases via SQL (i.e., SQL injection). Whole scripts written in Perl, Python and other languages can be injected into poorly designed Web applications and executed. Any time a Web application uses an interpreter of any type, there is danger of an injection attack. Let's look at the example of the most common injection flaw, SQL injection, which happens when an app uses user input without validation in the construction of an SQL query sent to the database. For example:

```
$query = "SELECT prodinfo FROM prodtable WHERE prodname = '" .
$_POST['prod_search'] . "'";
```

This query is in code for a product search; the user searches for a product name, and the product is displayed to the user. Here, the value for '*prod_search*' is supposed to be the user search term—which is not validated by the application. Now, to attack, an adversary may enter *'x' OR 'x'='x'*, which will reach the database as:

```
SELECT prodinfo FROM prodtable WHERE prodname = 'x' OR 'x'='x';
```

Now, logically, this query's WHERE filter always evaluates to true, thus returning the entire product database to the user. Besides this, an attacker could add data to the database (perform an INSERT in the injected SQL), modify data (an UPDATE statement) and may even delete the table or entire database.

Cross-site scripting (aka XSS)

Cross-site scripting (also known as XSS or CSS) occurs when dynamically generated Web pages display input that is not properly validated. This allows an attacker to embed malicious JavaScript code into the generated page, and execute the script on the machine of any user who views that site. Cross-site scripting could potentially impact any site that allows users to enter data without any validation. For more information and technical details on XSS, please go through *http://bit.ly/uiohTW*.

Cross-site request forgery (aka CSRF)

CSRF attacks are done using cross-site requests, where a page from Site A makes requests to another site for resources that are part of the page (like images, for example). It's a 'forgery' because it's invisible to the users. In a typical CSRF attack, the attacker creates a hidden HTTP request inside the victims' Web browser, which is executed in the victims' authentication context, without their knowledge.

The main aim of a CSRF attack is to perform an action against the target site using the victim's privileges in a properly authenticated session. The key requirement is the attacker's prior access to, and knowledge of, the Web application's valid URLs.

Insecure direct object reference

Insecure direct object reference occurs when the application fails to authorise a target object over the Web server, or in other words, when a Web application exposes an internal object or Web page, revealing sensitive information like database records, files or URLs. To elaborate, it is like a bank's application in which the admin can view details of all savings accounts in their region, and can also download it in Excel (.xls) format. Now, the URL for the Excel file could be:

```
http://www.mybank.com/admin/display?file=/admin/manage/
output07072012.xls
```

After the admin downloads, reads and deletes the file from the local system, before leaving, an adversary, getting access on the same machine, can view the above URL in browser history, and can repeat the request without even logging in. The adversary doesn't need to know the login credentials for administration—the attack succeeds because the object (.xls here) is referred without authorisation or authentication.

Parameter manipulation

Parameter manipulation, or parameter tampering, attacks occur when an adversary manipulates or tampers with request or response parameters to get an unauthorised or unauthenticated response. Parameter manipulation is usually done with cookies, form fields, URL query strings, or HTTP headers. For instance, when visiting a site like *http://www.fictitiousbank.com/* and accessing the *delete billers* page (*deletebillers.do*), the attacker captures a legitimate request to this page with an interceptor tool (such as Burp, Paros, etc), as shown below:

```
POST /billpay/deletebillers.do HTTP/1.1
Host: www.fictitiousbank.com
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; de; rv:1.9)
Gecko/2008052906 Firefox/3.6.2t
---
---
Cookie: MmdlkfhiefV52598542jrewr443
---
Content-Length: 12

userID=31385&requesttype=netbank
```

The attacker could change the *userID* in the above request to that of another customer: